

Proiect Programare Concurenta

Ioana Leuştean şi Bogdan Macovei

Proiectul presupune modelarea, implementarea şi testarea unei aplicaţii concurente, folosind şi tool-uri de Inteligenţă Artificială pentru generare de cod.

Reguli

- Proiectele vor fi realizate în echipe cu doi membri.
- Fiecare echipă va realiza un referat, care va documenta etapele proiectului, problemele şi soluţiile găsite. Referatele vor fi încărcate în Teams (aproximativ cu 2 zile înainte de prezentarea proiectului) şi vor fi verificate cu Turnitin.
- Proiectele vor fi prezentate oral pe slide-uri, iar prezentarea va fi facuta de ambii membri ai echipei.
- Prezentarea va avea loc in sesiune, dar este independentă de susţinerea examenului scris. Participarea la examen şi susținerea proiectului nu sunt condiţionate una de cealaltă.
- Punctajul maxim acordat pentru proiect este de 30 puncte.
- Realizarea şi prezentarea proiectului sunt obligatorii pentru a putea promova examenul. Studenţii care participă la examenul scris dar nu prezintă proiect vor avea în catalog nota ≤ 4 în sesiunea respectivă, dar punctajul obţinut la examenul scris poate fi păstrat în sesiunile ulterioare.

Descriere proiect

- (1) Echipa îşi va alege un program concurent. Două echipe diferite vor alege programe diferite. Găsiţi detalii privind alegerea proiectului în ultima secţiune.
- (2) Interacţiunea concurentă va fi modelata în PlusCal/ TLA+ si verificată folosind tool-ul dedicat.

Referinţe TLA+¹:

<https://lamport.azurewebsites.net/tla/tla.html>

<https://foundation.tlap1.us/>

<https://learntla.com/>

<https://github.com/tlaplus/>

¹Leslie Lamport, creatorul TLA+, este laureat al Premiului Turing (echivalentul premiului Nobel pentru Informatică) pentru contribuţii la dezvoltarea teoretică şi practică a sistemelor distribuite şi concurente. https://amturing.acm.org/award_winners/lamport_1205376.cfm

- (3) Pentru modelul verificat se va realiza o implementare completă. Se va folosi un tool IA pentru a genera cod din specificatia PlusCal/TLA+ în limbajele alese pentru implementare.
- (4) Fiecare implementare va fi testată cu tool-uri sau prin programe specifice în limbajul respectiv sau vă puteți genera propriile teste. Exemple de tool-uri:
 - **Java**
 JavaPathFinder pentru testare
<https://github.com/javapathfinder/jpf-core/wiki>
 - **Haskell:**
 HasCal pentru traducerea din PlusCal: <https://github.com/Gabriella439/HasCal>
 Modul pentru testare: <https://hackage.haskell.org/package/chp-plus-1.3.1.2/docs/Control-Concurrent-CHP-Test.html>
 - **Go:**
 PGo pentru traducerea din PlusCal: <https://github.com/DistCompiler/pgo>
 Go Race Detector pentru testare: <https://go.dev/blog/race-detector>
 - **Erlang:**
 Erla+ pentru traducerea din PlusCal: <https://dl.acm.org/doi/10.1145/3677995.3678190>, <https://github.com/mmhrstov/Erlapplus>
 CommonTest pentru testare: https://www.erlang.org/doc/apps/common_test/common_test_app.html,
<https://learnyouesomeerlang.com/common-test-for-uncommon-tests>
- (5) Pentru implementare se vor folosi doua limbaje dintre cele prezentate la curs (Java, Haskell, Go, Erlang), fiecare membru al echipei va prezenta câte o implementare.
- (6) Fiecare proces - modelare, interacțiune cu tool-ul de IA, implementare, testare, va fi documentată în referat, iar prezentarea va prezenta concluziile pentru fiecare etapă.
- (7) Fiecare proiect trebuie să prezinte și un exemplu în care tool-urile folosite depistează o greșeală și corectarea ei. Acest lucru se poate realiza prin documentarea greșelilor inițiale sau prin modificarea unui program corect și analiza modului în care metodele și tool-urile folosite depistează eroarea.

Alegerea proiectului

Pentru proiect puteți alege o specificație TLA+ deja existentă (A) sau puteți propune o implementare /idee proprie (B). **Indiferent dacă optați pentru (A) sau (B), veți discuta propunerea pe care o aveți cu Bogdan Macovei, iar după ce aveți acordul lui, veți înscrie echipa și proiectul într-un fisier ce va fi disponibil în Teams.**

Termen limită pentru alegerea proiectului: 18 aprilie 2026.

(A) Ca punct de plecare, puteți alege specificații din tutorialele de TLA+ sau din experiențele companiilor care au folosit TLA+, de exemplu:

<https://github.com/tlaplus/examples/>

<https://learntla.com/examples/index.html#pluscal-specs>
<https://lamport.azurewebsites.net/tla/industrial-use.html>
<https://assets.amazon.science/67/f9/92733d574c11ba1a11bd08bfb8ae/how-amazon-web-services-pdf>

După ce înțelegeți un astfel de exemplu, veți încerca să îl modificați (sa îl complicați, să introduceți o vulnerabilitate, etc.) urmînd pașii (1)-(7) de mai sus.

(B) Alternativ, puteți pleca de la un program sau de la o problemă concretă pe care să o specificați în TLA+ și să urmați pașii (1)-(7), comparînd eventual implementarea inițială cu cea obținută din specificația PlusCal.